

RETO TÉCNICO DBA SEMISENIOR

Presentado por:

Andrés Rico Quintero

Tecnologías:

- Windows 11
- WSL2 Ubuntu 24.04
- PostgreSQL 16
- pgAdmin 4

1. ADMINISTRACIÓN DE BASES DE DATOS

Antes de instalar la instancia de PostgreSQL, se escogió por implementar un entorno en Linux sin la necesidad de usar una maquina virtual tradicional.

Esto se hizo mediante WSL2 sobre Windows, en el PowerShell, lo que permitió disponer de un menor consumo de recursos y tiempos de configuración

Se procede a instalar el WSL, junto con el Ubuntu 24.04

```
Administrador: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\WINDOWS\system32> wsl --status
El Subsistema de Windows para Linux no está instalado. Para instalar, ejecute "wsl.exe --install".
Para obtener más información, visite https://aka.ms/wslinstall
PS C:\WINDOWS\system32> wsl --install
Descargando: Subsistema de Windows para Linux 2.7.8
[=====81.1%=====]
```

```
Selecciónar Administrador: Windows PowerShell
Versión predeterminada: 2
WSL1 no es compatible con la configuración actual del equipo.
Habilita el componente opcional "Subsistema de Windows para Linux" para usar WSL1.
PS C:\WINDOWS\system32> wsl --list --online
A continuación se muestra una lista de distribuciones válidas que se pueden instalar.
Instalar con "wsl.exe --install <Distro>"

NAME                FRIENDLY NAME
Ubuntu              Ubuntu
Ubuntu-26.04        Ubuntu 26.04 LTS
Ubuntu-24.04        Ubuntu 24.04 LTS
```

Además de, crear el usuario administrativo **dbaadmin**, que posteriormente fue utilizado para la administración del servidor de bases de datos.

```
Create a default Unix user account: dbaadmin
New password:
Retype new password:
passwd: password updated successfully
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

dbaadmin@Andres:/mnt/c/WINDOWS/system32$
```

Se valida:

```
dbaadmin@Andres:/mnt/c/WINDOWS/system32$ cd ~
dbaadmin@Andres:~$ pwd
/home/dbaadmin
dbaadmin@Andres:~$
```

1.1 Instalación de PostgreSQL

Se seleccionó PostgreSQL por ser uno de los motores relacionales de código abierto más utilizados en ambientes empresariales.

Se instaló PostgreSQL 16 utilizando el gestor de paquetes APT de Ubuntu

```
sudo apt install postgresql postgresql-contrib -y
```

Se verifica el servicio con:

```
sudo service postgresql status
```

```
Data page checksums are disabled.

fixing permissions on existing directory /var/lib/postgresql/16/main ... ok
creating subdirectories ... ok
selecting dynamic shared memory implementation ... posix
selecting default max_connections ... 100
selecting default shared_buffers ... 128MB
selecting default time zone ... America/Bogota
creating configuration files ... ok
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... ok
Setting up postgresql-contrib (16+257build1.1) ...
Setting up postgresql (16+257build1.1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
dbaadmin@Andres:~$ sudo service postgresql status
• postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: active (exited) since Wed 2026-06-24 15:10:01 -05; 38s ago
   Main PID: 2583 (code=exited, status=0/SUCCESS)
   CPU: 1ms

Jun 24 15:10:01 Andres systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
Jun 24 15:10:01 Andres systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
dbaadmin@Andres:~$
```

1.2 Crear la base de datos del reto

Se valida que el servidor de PostgreSQL quedo habilitado

```
dbaadmin@Andres:~$ sudo -u postgres psql
psql (16.14 (Ubuntu 16.14-0ubuntu0.24.04.1))
Type "help" for help.

postgres=#
```

Para la creación de la base de datos usaremos la herramienta administrativa pgAdmin; por lo que Se modificó el archivo **postgresql.conf** para permitir conexiones remotas.

```
dbaadmin@Andres:~$ psql --version
psql (PostgreSQL) 16.14 (Ubuntu 16.14-0ubuntu0.24.04.1)
dbaadmin@Andres:~$ sudo find /etc/postgresql -name postgresql.conf
/etc/postgresql/16/main/postgresql.conf
dbaadmin@Andres:~$ hostname -I
172.28.70.95
dbaadmin@Andres:~$
```

Cambiando el listen_adresses, por '*'

```
# - Connection Settings -

#listen_addresses = '*'          # what IP address(es) to listen on;
                                # comma-separated list of addresses;
                                # defaults to 'localhost'; use '*' for all
                                # (change requires restart)
port = 5432                      # (change requires restart)
max_connections = 100           # (change requires restart)
#reserved_connections = 0       # (change requires restart)
#superuser_reserved_connections = 3 # (change requires restart)
unix_socket_directories = '/var/run/postgresql' # comma-separated list of directories
                                # (change requires restart)
#unix_socket_group = ''         # (change requires restart)
#unix_socket_permissions = 0777 # begin with 0 to use octal notation
                                # (change requires restart)

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo     M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo     M-6 Copy
```

Además, de configurar el archivo **pg_hba.conf**.

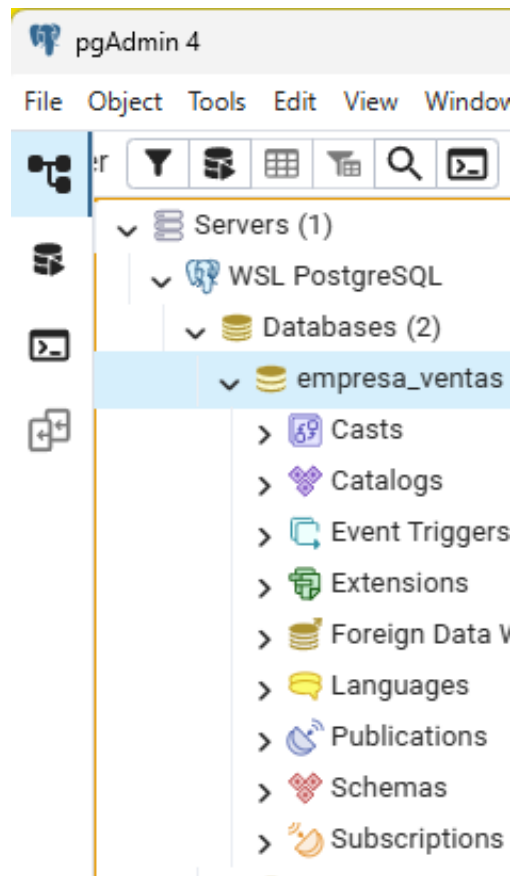
```
postgres=# ALTER USER postgres PASSWORD 'DBA2026*';
ALTER ROLE
postgres=# \q
dbaadmin@Andres:~$ sudo tail -n 20 /etc/postgresql/16/main/pg_hba.conf
# Noninteractive access to all databases is required during automatic
# maintenance (custom daily cronjobs, replication, and similar tasks).
#
# Database administrative login by Unix domain socket
local  all             postgres            peer

# TYPE  DATABASE      USER        ADDRESS              METHOD

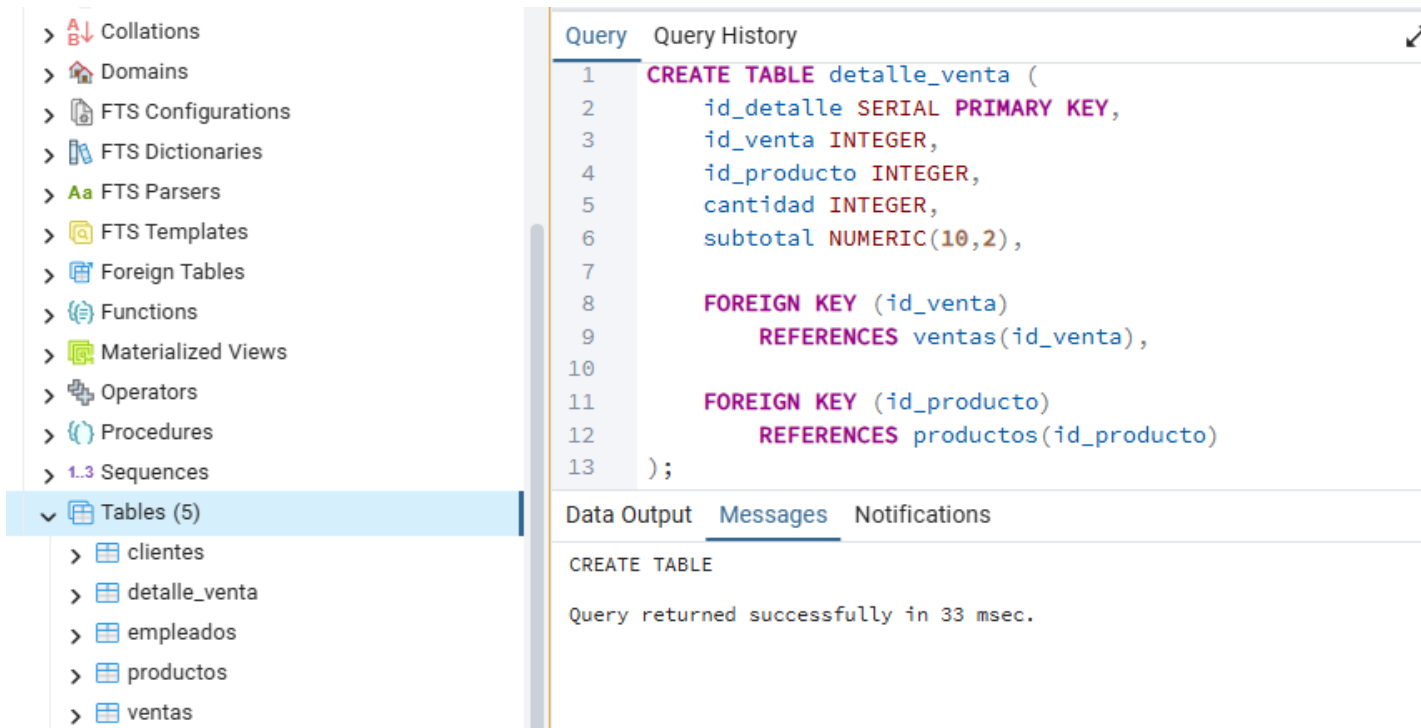
# "local" is for Unix domain socket connections only
local  all             all            peer
# IPv4 local connections:
host   all             all           127.0.0.1/32         scram-sha-256
# IPv6 local connections:
host   all             all           ::1/128              scram-sha-256
# Allow replication connections from localhost, by a user with the
# replication privilege.
local  replication     all            peer
host   replication     all           127.0.0.1/32         scram-sha-256
host   replication     all           ::1/128              scram-sha-256
host   all              all           172.28.70.0/24       scram-sha-256
dbaadmin@Andres:~$
```

Ahora, se crea la base de datos en PostgreSQL, para después obsérvala en pgAdmin

```
postgres=# CREATE DATABASE empresa_ventas;  
CREATE DATABASE  
postgres=# \l  
postgres=# \c empresa_ventas  
You are now connected to database "empresa_ventas" as user "postgres".  
empresa_ventas=#
```



Ya con la sesión abierta, se crearon las tablas desde la consola; para ello se diseñó un modelo de datos relacional para representar el proceso de ventas de una empresa, garantizando la integridad y consistencia de la información.



La base de datos, fue compuesta por cinco tablas (**clientes**, **empleados**, **productos**, **ventas** y **detalle_venta**); cabe mencionar que todas poseen una clave primaria que permite identificar cada registro de manera única. Garantizando unicidad de los registros, integridad de los datos, facilidad de búsqueda y la relación entre ellas.

Para garantizar la integridad referencial evitando registros huérfanos, en **detalle_venta** se implementaron dos claves foráneas, así como se observó en la última imagen; además la tabla **venta** también posee relaciones con **clientes** y **empleados**

Ahora, en la consola se agregan registros, para posteriormente ser consultados:

```
INSERT INTO clientes(nombre, correo, telefono)
VALUES
('Juan Pérez','juan@gmail.com','3001111111'),
('Ana Gómez','ana@gmail.com','3002222222'),
('Carlos López','carlos@gmail.com','3003333333');
```

```
INSERT INTO empleados(nombre, cargo)
VALUES
('Laura Díaz','Vendedor'),
('Andrés Ruíz','Supervisor');
```

```
INSERT INTO productos(nombre, precio, stock)
VALUES
('Laptop',3500,20),
('Mouse',50,100),
('Teclado',120,50),
('Monitor',800,15);
```

```
INSERT INTO ventas(fecha,id_cliente,id_empleado)
VALUES
('2026-06-24',1,1),
('2026-06-24',2,2);
```

```
INSERT INTO detalle_venta(id_venta,id_producto,cantidad,subtotal)
VALUES
(1,1,1,3500),
(1,2,2,100),
(2,3,1,120),
(2,4,1,800);
```

Se prueba con una consulta avanzada:

QueryQuery History↗Scratch Pad x

```
1 SELECT c.nombre AS cliente,
2       p.nombre AS producto,
3       dv.cantidad,
4       dv.subtotal,
5       e.nombre AS vendedor
6 FROM ventas v
7 JOIN clientes c
8     ON v.id_cliente = c.id_cliente
9 JOIN empleados e
10    ON v.id_empleado = e.id_empleado
11 JOIN detalle_venta dv
12    ON v.id_venta = dv.id_venta
13 JOIN productos p
14    ON p.id_producto = dv.id_producto;
```

Data OutputMessagesNotifications

≡+📄▼📋▼🗑️🗄️⬇️📶SQL

Showing rows: 1 to 4✎📧Page No: 1 of 1⏪⏩⏴⏵

	cliente character varying (100) 🔒	producto character varying (100) 🔒	cantidad integer 🔒	subtotal numeric (10,2) 🔒	vendedor character varying (100) 🔒
1	Juan Pérez	Laptop	1	3500.00	Laura Díaz
2	Juan Pérez	Mouse	2	100.00	Laura Díaz
3	Ana Gómez	Teclado	1	120.00	Andrés Ruiz
4	Ana Gómez	Monitor	1	800.00	Andrés Ruiz

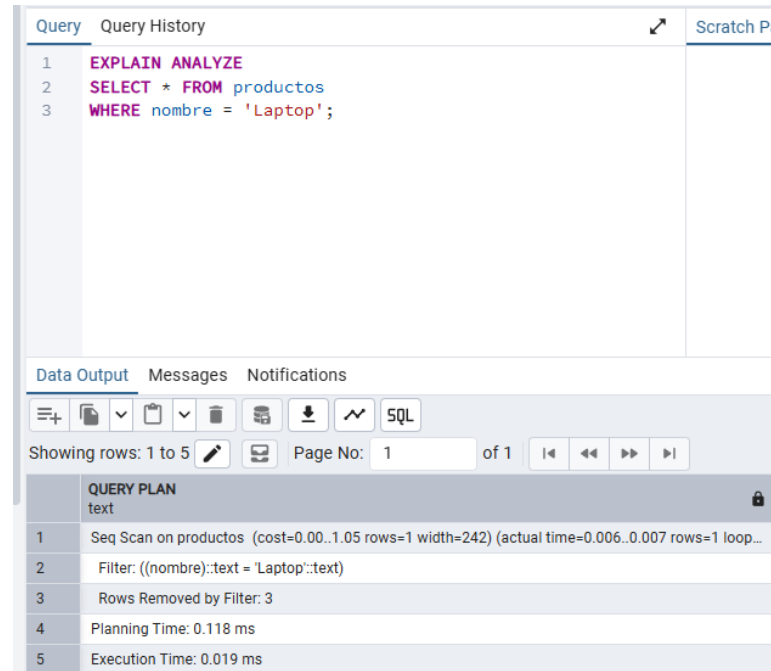
Ahora, pese a que la tabla contiene pocos registros y PostgreSQL utiliza un Seq Scan, se implementó índices siguiendo las buenas prácticas de diseño

Se creó en la consola:

```
CREATE INDEX idx_producto_nombre
```

```
ON productos(nombre);
```

Se corrobora:



The screenshot shows a PostgreSQL query editor interface. The top bar has tabs for 'Query', 'Query History', and 'Scratch Pad'. The 'Query' tab is active, displaying a SQL query:

```
1 EXPLAIN ANALYZE
2 SELECT * FROM productos
3 WHERE nombre = 'Laptop';
```

Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with 5 rows and 1 column. The table is titled 'QUERY PLAN' and contains the following data:

	QUERY PLAN
1	Seq Scan on productos (cost=0.00..1.05 rows=1 width=242) (actual time=0.006..0.007 rows=1 loop...)
2	Filter: ((nombre)::text = 'Laptop'::text)
3	Rows Removed by Filter: 3
4	Planning Time: 0.118 ms
5	Execution Time: 0.019 ms

Y como se menciona antes PostgreSQL eligió un Sequential Scan debido al pequeño tamaño de la tabla. En ambientes productivos con grandes volúmenes de datos, el optimizador utilizaría el índice para reducir el tiempo de búsqueda.

Para evitar que el inventario tenga valores negativos, se hace una restricción de integridad:
stock INTEGER CHECK (stock >= 0)

Esta restricción garantiza la consistencia de los datos almacenados.

Query

Query History

1

2

3

4

SELECT

conname,

pg_get_constraintdef(oid)

FROM pg_constraint

WHERE conrelid = 'productos'::regclass;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

Showing

	conname name	pg_get_constraintdef text
1	productos_stock_che...	CHECK ((stock >= 0))
2	productos_pkey	PRIMARY KEY (id_produc...

1.3 Gestión de usuarios y roles en una base de datos productiva

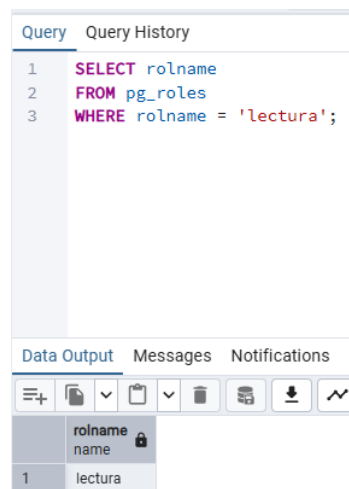
Se desea implementar un esquema de seguridad basado en el principio de mínimo privilegio, garantizando que cada usuario tenga únicamente los permisos necesarios para realizar sus funciones.

Se crearon roles independientes para separar las responsabilidades dentro del sistema:

- Rol de lectura.
- Rol de escritura.
- Usuario administrador PostgreSQL.

El rol de lectura únicamente posee permisos de consulta (SELECT), mientras que el rol de escritura cuenta con permisos de inserción, actualización y eliminación de registros. Las actividades administrativas permanecen restringidas al usuario administrador.

Esta separación reduce riesgos de modificaciones accidentales y fortalece la seguridad del sistema



```
Query  Query History
1  SELECT rolname
2  FROM pg_roles
3  WHERE rolname = 'lectura';
```

Data Output Messages Notifications

	rolname
1	lectura

Para monitorear las conexiones activas y las sesiones de usuarios se utilizaron las vistas del sistema:

The screenshot shows a database management interface with a 'Query' tab and a 'Scratch Pad' tab. The 'Query' tab contains a SQL query to select active sessions from the 'pg_stat_activity' table. The 'Data Output' tab shows the results of the query, which include columns for pid, username, state, and query. The results show five rows of data, including sessions for 'postgres' and 'active' states.

```
1 SELECT pid,
2     username,
3     state,
4     query
5 FROM pg_stat_activity;
```

	pid integer	username name	state text	query text
1	4083	[null]	[null]	
2	4084	postgres	[null]	
3	5800	postgres	idle	SELECT roles.oid as id, roles.rolname as name, roles.rolsuper as is_superuser, CASE
4	5801	postgres	idle	/*pga4dash*/ SELECT 'session_stats' AS chart_name, pg_catalog.row_to_json(t) AS chart_data FROM (SELEC
5	19409	postgres	active	SELECT pid,

Permitiendo al administrador identificar:

- usuarios conectados.
- consultas ejecutadas.
- sesiones inactivas.
- posibles bloqueos o consumos excesivos.

En la imagen se identificaron sesiones activas e inactivas del usuario postgres, permitiendo validar el estado operativo de la base de datos y realizar auditoría básica de accesos.

Ante la salida de un empleado o cambio de funciones, los permisos pueden revocarse mediante:

REVOKE ALL PRIVILEGES

ON ALL TABLES IN SCHEMA public

FROM lectura;

Posteriormente, el rol puede deshabilitarse o eliminarse del sistema.

1.4 Mantenimiento preventivo periódico de la base de datos

Para garantizar el correcto funcionamiento de la base de datos mediante actividades periódicas de mantenimiento orientadas a conservar el rendimiento, la disponibilidad y la integridad de la información.

Actualización de estadísticas

Se ejecutó el comando:

ANALYZE;

El objetivo de esta operación es actualizar las estadísticas utilizadas por el optimizador de consultas para generar planes de ejecución eficientes.

Reindexación

se realizó la reconstrucción de índices mediante:

REINDEX TABLE productos;

La reindexación permite reorganizar las estructuras de los índices, mejorando los tiempos de búsqueda y reduciendo la fragmentación.

Limpieza y optimización

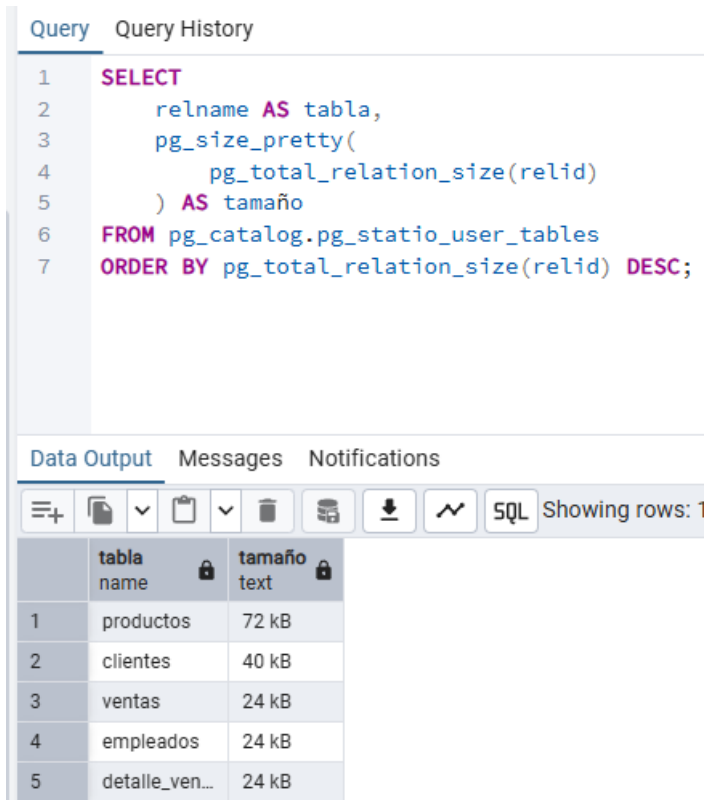
Se ejecutó:

VACUUM ANALYZE productos;

Este proceso libera espacio ocupado por registros obsoletos y actualiza las estadísticas de la tabla.

Monitoreo del crecimiento

Se consultó el tamaño de las tablas utilizando:



The screenshot shows a database query tool interface. At the top, there are tabs for 'Query' and 'Query History'. The 'Query' tab is active, displaying an SQL query. Below the query, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with 5 rows. The table has two columns: 'tabla name' and 'tamaño text'. The rows are numbered 1 to 5. The first row shows 'productos' with a size of '72 kB'. The second row shows 'clientes' with a size of '40 kB'. The third row shows 'ventas' with a size of '24 kB'. The fourth row shows 'empleados' with a size of '24 kB'. The fifth row shows 'detalle_ven...' with a size of '24 kB'.

```
1 SELECT
2     relname AS tabla,
3     pg_size_pretty(
4         pg_total_relation_size(relid)
5     ) AS tamaño
6 FROM pg_catalog.pg_statio_user_tables
7 ORDER BY pg_total_relation_size(relid) DESC;
```

	tabla name	tamaño text
1	productos	72 kB
2	clientes	40 kB
3	ventas	24 kB
4	empleados	24 kB
5	detalle_ven...	24 kB

Esta consulta permite identificar las tablas que presentan mayor crecimiento y facilita la planificación de capacidad.

2. SQL AVANZADO Y OPTIMIZACIÓN DE CONSULTAS

2.1 Consulta múltiples

Query

Query History

Scratch P

```
1 SELECT
2     c.nombre AS cliente,
3     p.nombre AS producto,
4     dv.cantidad,
5     dv.subtotal,
6     e.nombre AS vendedor
7 FROM ventas v
8 JOIN clientes c
9     ON v.id_cliente = c.id_cliente
10 JOIN empleados e
11     ON v.id_empleado = e.id_empleado
12 JOIN detalle_venta dv
13     ON v.id_venta = dv.id_venta
14 JOIN productos p
15     ON p.id_producto = dv.id_producto;
```

Data Output

Messages

Notifications

Showing rows: 1 to 4

Page No: 1

	cliente character varying (100)	producto character varying (100)	cantidad integer	subtotal numeric (10,2)	vendedor character varying (100)
1	Juan Pérez	Laptop	1	3500.00	Laura Díaz
2	Juan Pérez	Mouse	2	100.00	Laura Díaz
3	Ana Gómez	Teclado	1	120.00	Andrés Ruiz
4	Ana Gómez	Monitor	1	800.00	Andrés Ruiz

Se utilizaron múltiples JOIN para integrar la información distribuida en varias tablas relacionadas mediante claves foráneas.

Consulta implementada

Query

Query History

1

2

3

4

5

SELECT

nombre,

precio,

ROW_NUMBER() OVER (ORDER BY precio DESC) AS posicion

FROM productos;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Showing rows: 1 to 4

	nombre character varying (100)	precio numeric (10,2)	posicion bigint
1	Laptop	3500.00	1
2	Monitor	800.00	2
3	Teclado	120.00	3
4	Mouse	50.00	4

La función ROW_NUMBER genera una numeración secuencial sin agrupar los registros, permitiendo establecer rankings.

Subconsulta

Query	Query History
1	SELECT
2	nombre,
3	precio
4	FROM productos
5	WHERE precio > (
6	SELECT AVG(precio)
7	FROM productos
8	);

Data Output	Messages	Notifications
		
	nombre character varying (100) 	precio numeric (10,2) 
1	Laptop	3500.00

La subconsulta calcula el precio promedio de todos los productos y posteriormente compara cada registro.

2.2 Procedimiento almacenado

Se automatiza el cálculo del total de compras realizadas por un cliente, esto es importante para, generación de deportes, análisis comercial, segmentación de clientes, etc.

Se desarrolló el procedimiento almacenado:

sp_total_cliente

El procedimiento recibe:

- el identificador del cliente.
- una variable de salida con el total calculado

```
1 CREATE OR REPLACE PROCEDURE sp_total_cliente(  
2     IN cliente_id INTEGER,  
3     INOUT total NUMERIC  
4 )  
5 LANGUAGE plpgsql  
6 AS $$  
7 BEGIN  
8  
9     SELECT COALESCE(SUM(dv.subtotal),0)  
10    INTO total  
11    FROM ventas v  
12   JOIN detalle_venta dv  
13      ON v.id_venta = dv.id_venta  
14      WHERE v.id_cliente = cliente_id;  
15  
16 EXCEPTION  
17     WHEN OTHERS THEN  
18         total := -1;  
19  
20 END;  
21 $$;
```

Data Output Messages Notifications

CREATE PROCEDURE

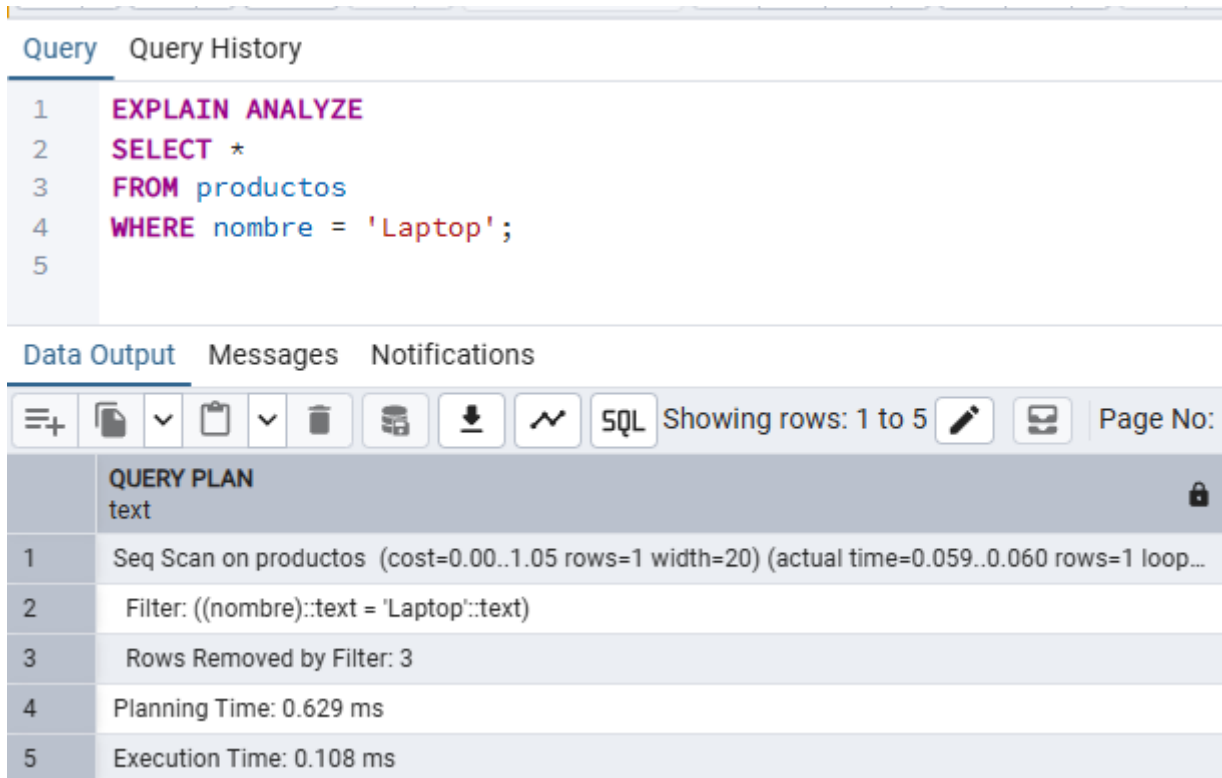
Query returned successfully in 39 msec.

El manejo de errores de *EXCEPTION*, Permite controlar errores durante la ejecución y devolver un valor de error.

Ahora, se comprueba:

1	CALL sp_total_cliente(1,0);
Data Output Messages Notifications	
	total numeric
1	3600.00

2.3 Identificación y optimización de una consulta



The screenshot displays a PostgreSQL query editor interface. At the top, there are tabs for 'Query' and 'Query History'. The 'Query' tab is active, showing a SQL query with line numbers 1 through 5. Below the query, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a toolbar with various icons and a 'Page No:' field. Below the toolbar, there is a table titled 'QUERY PLAN' with a lock icon. The table has two columns: a line number and a description of the query plan step.

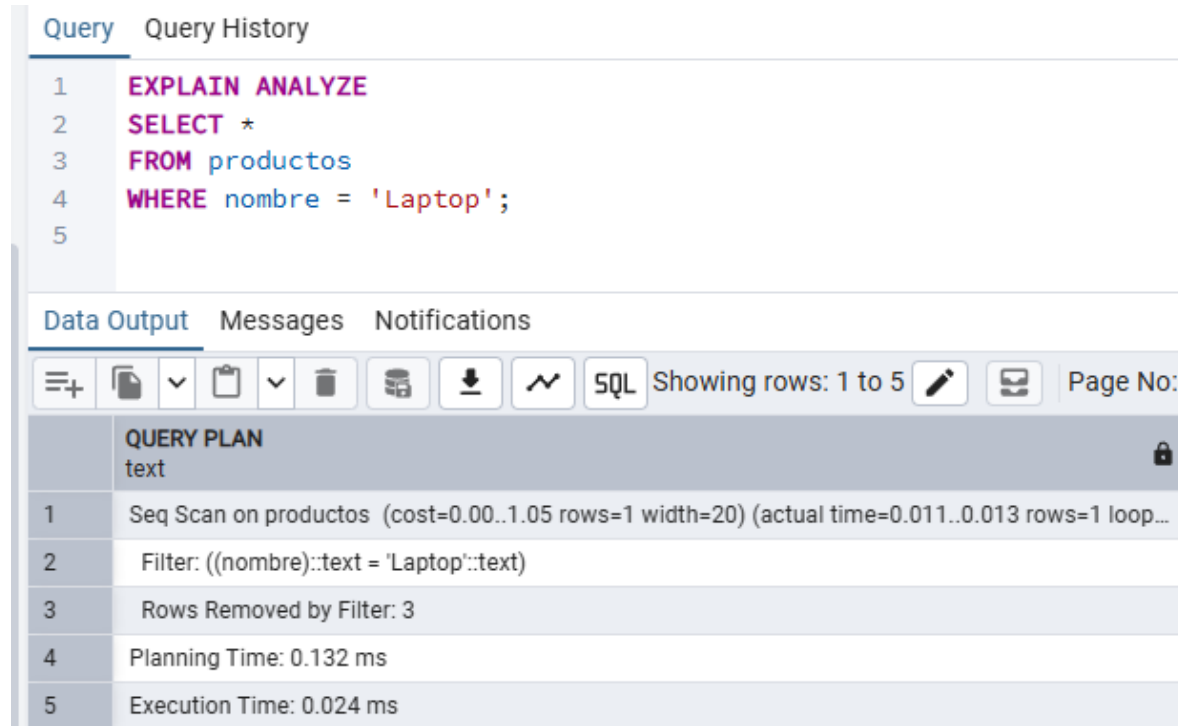
```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM productos
4 WHERE nombre = 'Laptop';
5
```

	QUERY PLAN
1	Seq Scan on productos (cost=0.00..1.05 rows=1 width=20) (actual time=0.059..0.060 rows=1 loop...
2	Filter: ((nombre)::text = 'Laptop'::text)
3	Rows Removed by Filter: 3
4	Planning Time: 0.629 ms
5	Execution Time: 0.108 ms

Se observó un escaneo secuencial (Sequential Scan), lo que significa que PostgreSQL recorrió toda la tabla para localizar el registro solicitado.

Aunque la consulta no presenta un problema de rendimiento debido al pequeño volumen de datos, en tablas con millones de registros un escaneo secuencial podría generar degradación del rendimiento.

Para mejorarlo, se aplica el índice sobre el campo de nombre:



The screenshot displays a database management interface. At the top, there are tabs for 'Query' and 'Query History'. The 'Query' tab is active, showing a SQL query with line numbers 1 through 5. Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a toolbar with various icons and a 'SQL' button. Below the toolbar, there is a 'QUERY PLAN' section with a lock icon. The query plan is displayed in a table with 5 rows.

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM productos
4 WHERE nombre = 'Laptop';
5
```

	QUERY PLAN
1	Seq Scan on productos (cost=0.00..1.05 rows=1 width=20) (actual time=0.011..0.013 rows=1 loop...
2	Filter: ((nombre)::text = 'Laptop'::text)
3	Rows Removed by Filter: 3
4	Planning Time: 0.132 ms
5	Execution Time: 0.024 ms

Como ya se había mencionado antes, el optimizador de PostgreSQL selecciona el plan de ejecución con menor costo. Debido al pequeño volumen de datos presente en la tabla, el motor determinó que el recorrido secuencial era más eficiente que acceder al índice

2.4 Atención de incidentes de rendimiento en producción

Paso 1: Validar la situación

Se debe confirmar:

- usuarios afectados.
- consultas involucradas.
- momento del incidente.
- impacto sobre la operación.

Paso 2: Monitorear las sesiones activas

```
SELECT pid,
```

```
    username,
```

```
    state,
```

```
    query
```

```
FROM pg_stat_activity;
```

Esta consulta permite identificar:

- consultas activas.
- sesiones bloqueadas.
- conexiones inactivas.

Paso 3: Analizar la consulta

Se ejecuta:

EXPLAIN ANALYZE

para conocer:

costos.

índices utilizados.

escaneos secuenciales.

operaciones costosas.

Paso 4: Revisar el consumo del sistema

Se analiza:

- CPU.
- memoria.
- disco.
- conexiones.

Utilizando:

- pgAdmin Dashboard.
- pg_stat_activity.
- pg_stat_database.

Paso 5: Implementar mejoras

- Las acciones pueden incluir:
- creación de índices.
- actualización de estadísticas.
- VACUUM ANALYZE.
- optimización de consultas.
- revisión de transacciones.

Coordinación con el equipo de desarrollo

- Las modificaciones se validan inicialmente en ambientes de pruebas para evitar impactos sobre producción.
- Los cambios se implementan durante ventanas de mantenimiento controladas y se documentan para garantizar la continuidad operativa.

3. BACKUP, RECOVERY Y CONTINUIDAD OPERATIVA

3.1 Backup

```
dbaadmin@Andres:~$ pg_dump -U postgres empresa_ventas > backup.sql
pg_dump: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432" failed: FATAL: Peer authentication failed for user "postgres"
dbaadmin@Andres:~$ sudo -u postgres pg_dump empresa_ventas > backup.sql
[sudo] password for dbaadmin:
dbaadmin@Andres:~$ ls -lh backup.sql
-rw-r--r-- 1 dbaadmin dbaadmin 11K Jun 24 17:34 backup.sql
dbaadmin@Andres:~$ head backup.sql
--
-- PostgreSQL database dump
--

\restrict jM5c34ymxpARFTd1byqsGpCvalgGG7NnWB10mxbyqZK4UuZmkBsfmjcGmJoTpST

-- Dumped from database version 16.14 (Ubuntu 16.14-0ubuntu0.24.04.1)
-- Dumped by pg_dump version 16.14 (Ubuntu 16.14-0ubuntu0.24.04.1)

SET statement_timeout = 0;
```

Se realiza el script

```
dbaadmin@Andres: ~
GNU nano 7.2 backup.sh *
#!/bin/bash

FECHA=$(date +%Y%m%d_%H%M)

sudo -u postgres pg_dump empresa_ventas \
> /home/dbaadmin/backup_$FECHA.sql

echo "Backup realizado correctamente: backup_$FECHA.sql"
```

```

dbaadmin@Andres:~$ nano backup.sh
dbaadmin@Andres:~$ chmod +x backup.sh
dbaadmin@Andres:~$ ./backup.sh
Backup realizado correctamente: backup_20260624_1737.sql
dbaadmin@Andres:~$ ls -lh backup_*.sql
-rw-r--r-- 1 dbaadmin dbaadmin 11K Jun 24 17:37 backup_20260624_1737.sql

```

Se programa con cron

```

dbaadmin@Andres:~$ crontab -e
no crontab for dbaadmin - using an empty one

Select an editor. To change later, run 'select-editor'.
 1. /bin/nano      <---- easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny
 4. /bin/ed

Choose 1-4 [1]: 0 2 * * * /home/dbaadmin/backup.sh
Choose 1-4 [1]:

```

Permisos

```

dbaadmin@Andres:~$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
0 2 * * * /home/dbaadmin/backup.sh
dbaadmin@Andres:~$

```

3.2 Escenario de fallo

Query

Query History

1

DROP TABLE productos CASCADE;

Data Output

Messages

Notifications

NOTICE: drop cascades to 2 other objects

DROP TABLE

Query returned successfully in 29 msec.

Query

Query History

Scratch Pad x

1

SELECT * FROM productos;

Data Output

Messages

Notifications

ERROR: relation "productos" does not exist

LINE 1: SELECT * FROM productos;

^

SQL state: 42P01

Character: 15

Total rows:

Query complete 00:00:00.023

CRLF

Ln 1, Col 25

ESP LAA

06:07 p. m.
24/06/2026

```
dbaadmin@Andres:~$ sudo -u postgres psql empresa_ventas < backup.sql
[sudo] password for dbaadmin:
SET
SET
SET
SET
SET
set_config
```

Se verifica

1 `SELECT * FROM productos;`

Data Output Messages Notifications

SQL Showing rows: 1 to 4 of 1

	Id_producto [PK] integer	nombre character varying (100)	precio numeric (10,2)	stock integer
1	1	Laptop	3500.00	20
2	2	Mouse	50.00	100
3	3	Teclado	120.00	50
4	4	Monitor	800.00	15

Total rows: 4 Query complete 00:00:00.036 CRLF Ln 1, Col 25



ESP LAA

06:10 p. m.
24/06/2026

Recovery Time Objective (RTO)

El RTO representa el tiempo máximo permitido para restaurar el servicio luego de una falla.

Durante la simulación de recuperación:

- Hora de falla: 6:07pm
- Inicio de restauración: 6:09pm
- Finalización: 6:10pm
- **RTO alcanzado: 1 minuto.**

Recovery Point Objective (RPO)

El RPO representa la cantidad máxima de información que podría perderse.

La estrategia implementada ejecuta un respaldo diario a las 02:00 AM.

Si ocurre una falla a las 05:00 PM:

- Último backup: 02:00 AM.
- Pérdida potencial: 15 horas.

RPO alcanzado: 15 horas.

3.3 ¿Cómo probar regularmente los respaldos?

Los respaldos deben restaurarse periódicamente en ambientes de prueba para validar su integridad. Se recomienda ejecutar restauraciones mensuales, verificar la consistencia de las tablas y documentar los tiempos de recuperación.

Diferencia entre backup válido y utilizable

Backup válido	Backup utilizable
El archivo existe	Puede restaurarse exitosamente
No presenta errores	Los datos son consistentes
Se genera correctamente	Cumple RTO y RPO

3.4 Conceptos

¿Qué es Alta Disponibilidad (HA)?

La Alta Disponibilidad (High Availability) es un conjunto de técnicas y arquitecturas diseñadas para garantizar que la base de datos continúe operando incluso cuando ocurre una falla del servidor principal

¿Qué es Failover?

El failover es el proceso mediante el cual un servidor secundario toma el control cuando el servidor principal presenta una falla.

Replicación Streaming

La replicación física transmite los archivos WAL del servidor principal hacia el servidor secundario.

Replicación lógica

Replica:

- tablas específicas.
- bases específicas.
- conjuntos de datos.

Permite:

- diferentes versiones.
- diferentes estructuras.
- migraciones.

Clúster Activo-Pasivo

Funcionamiento

- Solo un servidor atiende usuarios.
- El secundario espera.
- Si falla el principal, el secundario asume.

Ventajas

- Fácil administración.
- Alta disponibilidad.
- Menor complejidad.

Clúster Activo- Activo

Funcionamiento

- Balanceo de carga.
- Mayor disponibilidad.

Ventajas

- Mayor complejidad.
- Posibles conflictos de escritura.
- Administración más difícil.

Implementación básica en PostgreSQL

Para implementar un esquema básico de replicación en PostgreSQL se configuraría un servidor principal encargado de las operaciones de lectura y escritura y un servidor secundario destinado a recibir los registros WAL generados por el servidor principal. En el servidor primario se habilitarían parámetros como **wal_level**, **max_wal_senders** y **hot_standby**, además de crear un usuario con permisos de replicación. Posteriormente, se realizaría una copia inicial de la base de datos utilizando la herramienta **pg_basebackup**, permitiendo que el servidor secundario mantenga una sincronización continua con el nodo principal.

4. MONITOREO Y OBSERVABILIDAD

4.1 Implementación de la solución de monitoreo

Para el monitoreo de la base de datos se utilizaron herramientas nativas de PostgreSQL complementadas con las capacidades gráficas de pgAdmin 4. La primera herramienta empleada fue la vista del sistema `pg_stat_activity`, la cual permitió supervisar las conexiones activas, el estado de las sesiones y las consultas ejecutadas por los usuarios.

Query

Query History

Scratch Pad

1

2

3

4

5

```
SELECT pid,
      username,
      state,
      query
FROM pg_stat_activity;
```

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 8

Page No: 1 of 1

pid

integer

username

name

state

text

query

text

1

4083

[null]

[null]

2

4084

postgres

[null]

3

5800

postgres

idle

SELECT roles.oid as id, roles.rolname as name, roles.rolsuper as is_superuser, CASE V

4

5801

postgres

idle

/*pga4dash*/ SELECT 'session_stats' AS chart_name, pg_catalog.row_to_json(t) AS chart_data FROM (SELECT

5

19409

postgres

active

SELECT pid,

6

4080

[null]

[null]

7

4079

[null]

[null]

8

4082

[null]

[null]

4.2 Métricas claves

Estado de las conexiones

Query

Query History

1

2

3

4

SELECT state,

COUNT(*)

FROM pg_stat_activity

GROUP BY state;;

Data Output

Messages

Notifications

















state

count

text

bigint

1

2

3

[null]

active

idle

5

1

2

Monitoreo del crecimiento de las tablas

Query

Query History

1

2

3

4

5

6

SELECT

relname,

pg_size_pretty(

pg_total_relation_size(relid)

)

FROM pg_statio_user_tables;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑

🗄

⬇

⤿

SQL

Showing

relname

name

🔒

pg_size_pretty

text

🔒

1

clientes

40 kB

2

ventas

24 kB

3

empleados

24 kB

4

detalle_ven...

24 kB

5

productos

72 kB

Estadísticas operativas de la base

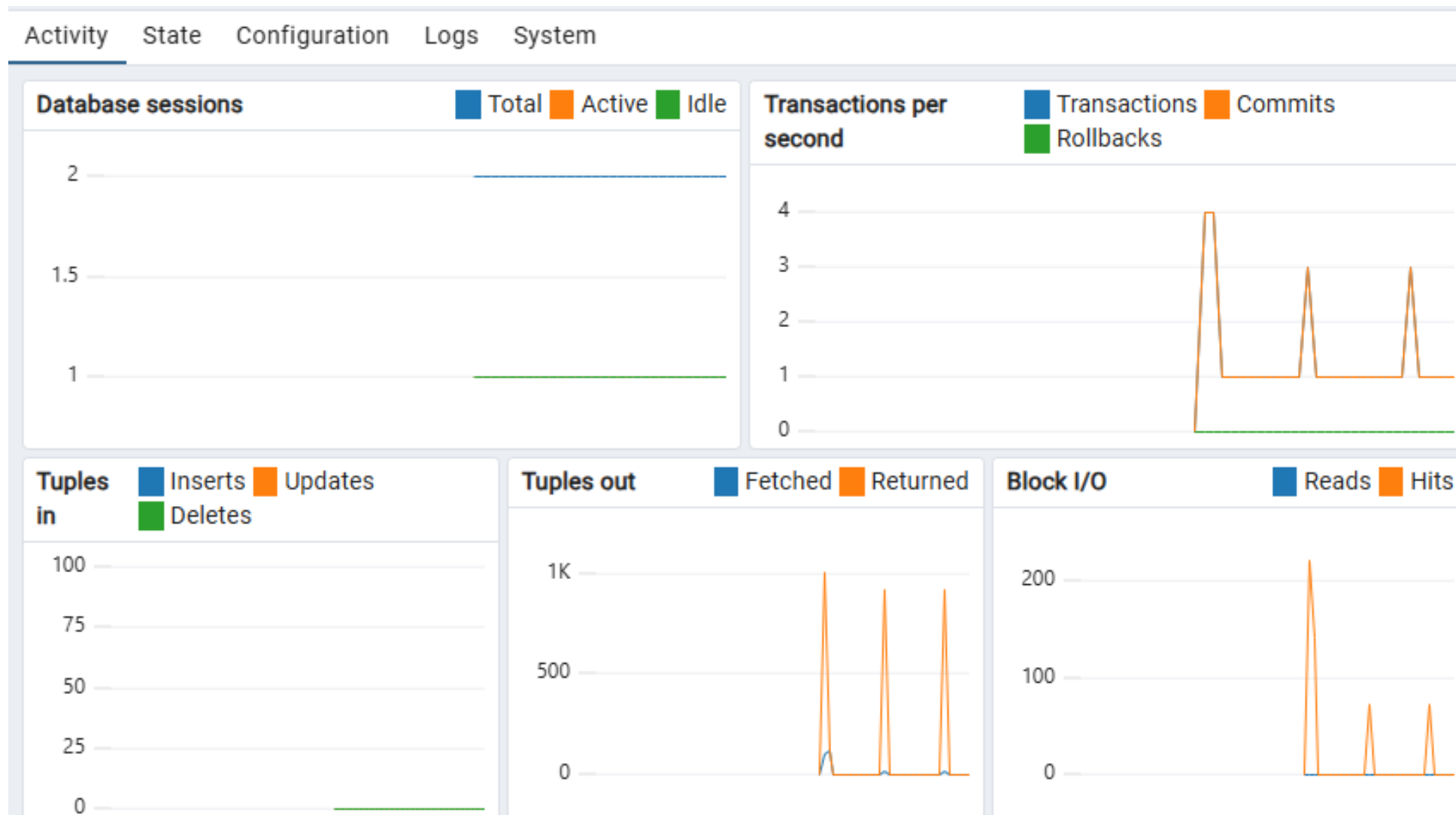
Query Query History

```
1  SELECT
2      datname,
3      numbackends,
4      xact_commit,
5      blks_read,
6      blks_hit
7  FROM pg_stat_database;
```

Data Output Messages Notifications

	datname name	numbackends integer	xact_commit bigint	blks_read bigint	blks_hit bigint
1	[null]	0	0	105	132083
2	postgres	1	2032	414	75941
3	template1	0	2805	964	164568
4	template0	0	0	0	0
5	empresa_vent...	2	5285	761	152637

Dashboard de monitoreo



4.3 Configuración de una alerta automática

Se implementó una alerta basada en el número de conexiones activas de la base de datos. La consulta evalúa el porcentaje de utilización de las conexiones configuradas y genera un estado de alerta cuando se supera el umbral definido. Esta estrategia permite detectar problemas de saturación antes de afectar la disponibilidad del servicio.

Query

Query History

1

SELECT

2

COUNT(*) AS conexiones,

3

CASE

4

WHEN COUNT(*) >= 80 THEN

5

'ALERTA: LIMITE DE CONEXIONES'

6

ELSE

7

'ESTADO NORMAL'

8

END AS estado

9

FROM pg_stat_activity;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Showing row

	conexiones bigint	estado text
1	8	ESTADO NORM...

4.4 Automatización de la respuesta ante picos de carga

Algunas actividades pueden ejecutarse automáticamente para reducir el impacto de los picos de carga. PostgreSQL incorpora el proceso Autovacuum, encargado de liberar espacio y mantener actualizadas las estadísticas utilizadas por el optimizador de consultas.

También es posible automatizar la identificación de consultas de larga duración o sesiones que permanezcan activas durante períodos excesivos. Esto permite reducir el impacto sobre el resto de los usuarios y mantener la estabilidad del sistema.

Query

Query History

1

SELECT

2

pid,

3

username,

4

now() - query_start AS duracion,

5

state,

6

query

7

FROM pg_stat_activity

8

WHERE state = 'active';

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Showing

	pid integer	username name	duracion interval	state text	query text
1	19409	postgres	00:00:00	active	SELECT

5. AUTOMATIZACIÓN Y SCRIPTING

5.1 script automatizado

Se desarrolló un script en Bash con el objetivo de automatizar tareas administrativas de la base de datos. El script realiza la generación del respaldo de la base de datos, registra la ejecución en un archivo de log y genera un reporte del tamaño de las tablas del sistema.

```
dbaadmin@Andres:~$ nano backup.sh
```

El respaldo de la base de datos se realiza mediante la herramienta *pg_dump*, generando archivos con fecha y hora para facilitar la identificación de los respaldos.

```
GNU nano 7.2
```

```
#!/bin/bash
```

```
FECHA=$(date +%Y%m%d_%H%M)
```

```
sudo -u postgres pg_dump empresa_ventas \  
> /home/dbaadmin/backup_$(FECHA).sql
```

```
echo "$FECHA Backup exitoso" \  
>> /home/dbaadmin/backup.log
```

Adicionalmente, el script puede generar un reporte del tamaño de las tablas para apoyar las actividades de monitoreo y capacidad del sistema.

```
dbaadmin@Andres:~$ sudo -u postgres psql -d empresa_ventas -c "
SELECT relname,
       pg_size_pretty(
         pg_total_relation_size(relid)
       )
FROM pg_statio_user_tables;"
[sudo] password for dbaadmin:
  relname      | pg_size_pretty
-----+-----
 clientes      | 40 kB
 ventas        | 24 kB
 empleados     | 24 kB
 detalle_venta | 24 kB
 productos     | 72 kB
(5 rows)
```

5.2 Programación del job

La ejecución automática del script se configuró mediante *cron*, permitiendo ejecutar las tareas diariamente a las 2:00 a.m.

```
dbaadmin@Andres:~$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
0 2 * * * /home/dbaadmin/backup.sh
dbaadmin@Andres:~$
```

La correcta ejecución de la tarea programada puede verificarse mediante la revisión del archivo de logs y la validación de los archivos de respaldo generados. La presencia de registros con fecha y hora confirma que el proceso se ejecutó exitosamente.

```
dbaadmin@Andres:~$ cat /home/dbaadmin/backup.log
20260624_2119 Backup exitoso
20260626_0200 Backup exitoso
```

En caso de fallo se modificaría el script:

```
#!/bin/bash
```

```
FECHA=$(date +%Y%m%d_%H%M)
```

```
if sudo -u postgres pg_dump empresa_ventas \  
> /home/dbaadmin/backup_ $FECHA.sql  
then  
echo "$FECHA Backup exitoso" \  
>> /home/dbaadmin/backup.log  
else  
echo "$FECHA ERROR EN BACKUP" \  
>> /home/dbaadmin/backup.log
```

5.3 Integración con CI/CD

Los scripts de administración de bases de datos pueden integrarse dentro de un pipeline de integración y despliegue continuo. Cuando un desarrollador realiza un cambio sobre el esquema de la base de datos, el código se almacena en un repositorio Git, donde se ejecutan validaciones automáticas antes del despliegue.

Posteriormente, las migraciones son ejecutadas en un ambiente de pruebas, donde se valida la sintaxis SQL, la integridad del esquema y el impacto sobre las consultas existentes. Una vez aprobadas las pruebas, los cambios se despliegan en producción durante una ventana controlada.

En caso de presentarse errores durante la implementación, el proceso de rollback permite restaurar la versión anterior del esquema utilizando scripts de reversión o respaldos previamente generados. Este enfoque reduce el riesgo operativo y garantiza la estabilidad de la base de datos.

6. SEGURIDAD Y AUDITORÍA

6.1 Implementación de buenas prácticas de seguridad

Se implementaron contraseñas robustas para los usuarios de PostgreSQL, utilizando combinaciones de letras, números y caracteres especiales. Esto reduce el riesgo de accesos no autorizados y fortalece la autenticación del sistema.

```
Create a default Unix user account: dbaadmin
New password:
Retype new password:
passwd: password updated successfully
```

El acceso remoto a la base de datos se restringió mediante el archivo *pg_hba.conf*, permitiendo conexiones únicamente desde la red autorizada del entorno WSL. Esta medida limita el acceso a direcciones IP específicas y reduce la superficie de ataque.

```
dbaadmin@Andres:~$ sudo tail -n 20 /etc/postgresql/16/main/pg_hba.conf
# Noninteractive access to all databases is required during automatic
# maintenance (custom daily cronjobs, replication, and similar tasks).
#
# Database administrative login by Unix domain socket
local    all             postgres                                peer
#
# TYPE      DATABASE      USER      ADDRESS              METHOD
# "local" is for Unix domain socket connections only
local    all             all                peer
# IPv4 local connections:
host     all             all          127.0.0.1/32          scram-sha-256
# IPv6 local connections:
host     all             all          ::1/128               scram-sha-256
# Allow replication connections from localhost, by a user with the
# replication privilege.
local    replication     all                peer
host     replication     all          127.0.0.1/32          scram-sha-256
host     replication     all          ::1/128               scram-sha-256
host     all              all          172.28.70.0/24         scram-sha-256
```


La separación de funciones se implementó mediante la creación de roles de lectura y escritura, aplicando el principio de mínimo privilegio. De esta forma, cada usuario dispone únicamente de los permisos necesarios para realizar sus actividades.

El enmascaramiento de datos sensibles puede implementarse ocultando parcialmente la información mostrada a determinados usuarios. Esto permite proteger datos personales o confidenciales sin afectar las operaciones del sistema.

1
2
3

```
SELECT
    LEFT(nombre,2) || '***'
FROM clientes;
```

Data OutputMessagesNotifications

≡+

▼

▼

	?column? text
1	Ju***
2	An***
3	Ca***

6.2 Auditoría y registros

1	<code>SHOW ssl;</code>
Data Output	
Messa	
≡+ ▼ ▼	
	ssl text 
1	on

Los registros de auditoría permiten identificar accesos, modificaciones y actividades sospechosas dentro de la base de datos. PostgreSQL permite registrar eventos relacionados con conexiones, cambios de esquema y operaciones administrativas.

Los siguientes parámetros pueden habilitarse en el archivo *postgresql.conf*

```
dbaadmin@Andres:~$ sudo grep -n "logging" /etc/postgresql/16/main/postgresql.conf
[sudo] password for dbaadmin:
459:                                # logging_collector to be on.
461:# This is used when logging to stderr:
462:#logging_collector = off          # Enable capturing of stderr, jsonlog,
467:# These are only used if logging_collector is on:
488:# These are relevant when logging to syslog:
494:# This is only relevant when logging to eventlog (Windows):
597:#log_parameter_max_length = -1    # when logging statements, limit logged
600:#log_parameter_max_length_on_error = 0    # when logging an error, limit logged
```

Ante un incidente de acceso no autorizado, los registros pueden consultarse para identificar intentos fallidos de conexión, usuarios involucrados y fechas del incidente.

Ejemplo de consulta:

Query

Query History

Scratch Pad

1

2

```
SELECT *
FROM pg_stat_activity;
```

Data Output

Messages

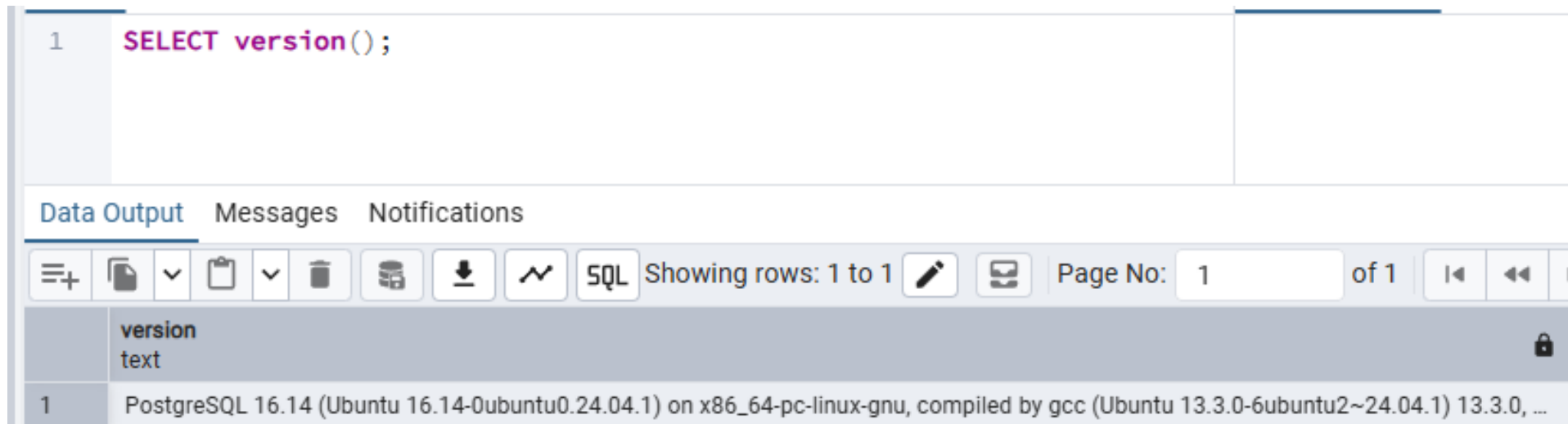
Notifications

6.3 Gestión de vulnerabilidades críticas

Ante la publicación de una vulnerabilidad crítica del motor de base de datos, el primer paso consiste en evaluar el impacto sobre la versión instalada y determinar si el entorno se encuentra afectado. Posteriormente se analizan los riesgos y se define una estrategia de mitigación.

Mientras se aplica el parche, pueden implementarse controles compensatorios como la restricción de accesos, el bloqueo de conexiones externas, la limitación de privilegios y el incremento del monitoreo sobre el servidor.

Finalmente, la actualización se ejecutaría durante una ventana de mantenimiento previamente programada. Antes de aplicar el parche se realizaría un respaldo completo de la base de datos y posteriormente se validaría el correcto funcionamiento del sistema, minimizando así el impacto sobre la operación.



The screenshot displays a database management interface. At the top, a query editor shows the SQL command `1 SELECT version();`. Below the editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with one row of results. The table has a single column labeled 'version' with a data type of 'text'. The result text is: 'PostgreSQL 16.14 (Ubuntu 16.14-0ubuntu0.24.04.1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0, ...'. The interface includes various icons for file operations, a status bar indicating 'Showing rows: 1 to 1', and a 'Page No: 1 of 1'.

version
PostgreSQL 16.14 (Ubuntu 16.14-0ubuntu0.24.04.1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0, ...

7. SOPORTE 24/7 E INCIDENCIAS CRÍTICAS

7.1 Manejo de una incidencia crítica fuera del horario laboral

Ante una caída de la base de datos fuera del horario laboral, el primer paso sería confirmar el alcance del incidente y validar si el servicio se encuentra completamente inaccesible o si únicamente afecta a determinados usuarios. Posteriormente se revisaría el estado del servicio PostgreSQL, los logs del sistema y las conexiones activas para determinar la causa del problema.

```
dbaadmin@Andres:~$ sudo service postgresql status
• postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: active (exited) since Wed 2026-06-24 16:56:39 -05; 1 day 9h ago
     Main PID: 4096 (code=exited, status=0/SUCCESS)
        CPU: 825us

Jun 24 16:56:39 Andres systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
Jun 24 16:56:39 Andres systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
```

Si la causa corresponde a una pérdida masiva de conexiones, se analizarían las sesiones activas mediante las vistas del sistema para identificar bloqueos, consultas problemáticas o saturación del servidor.

Query

Query History

Scratch Pad x

1

2

3

4

5

SELECT

pid,

username,

state,

query

FROM

pg_stat_activity;

Data Output

Messages

Notifications

Showing rows: 1 to 8

Page No: 1 of 1

	pid integer	username name	state text	query text
1	4083	[null]	[null]	
2	4084	postgres	[null]	
3	5800	postgres	idle	SELECT r.oid, r.rolname, r.rolcanlogin, r.rolsuper, d.description FROM pg_catalog.pg_roles r LEFT JOIN pg_ca
4	5801	postgres	idle	/*pga4dash*/ SELECT 'session_stats' AS chart_name, pg_catalog.row_to_json(t) AS chart_data FROM (SELECT
5	19409	postgres	active	SELECT pid,
6	4080	[null]	[null]	
7	4079	[null]	[null]	
8	4082	[null]	[null]	

Como ejemplo, podría presentarse una situación en la cual la base de datos deja de responder debido a un número excesivo de conexiones activas. En este caso se identificarían las sesiones responsables, se liberarían los recursos afectados y se restablecería el servicio, documentando posteriormente todas las acciones realizadas.

7.2 Atención de un deadlock o consulta de larga duración

Ante un bloqueo o una consulta de larga duración, el primer paso consiste en detectar las sesiones involucradas y determinar cuál de ellas está afectando la operación del sistema. El monitoreo de las sesiones activas permite identificar consultas bloqueadas y procesos que consumen recursos durante períodos prolongados.

Query

Query History

1

SELECT

2

pid,

3

username,

4

now() - query_start AS duracion,

5

state,

6

query

7

FROM pg_stat_activity

8

WHERE state='active';

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing row

	pid integer	username name	duracion interval	state text	query text
1	19409	postgres	00:00:00	active	SELECT

Posteriormente se analiza el impacto de la consulta sobre los usuarios y se determina si es necesario finalizar la sesión o esperar la finalización del proceso. Toda intervención debe ser comunicada al equipo de desarrollo y a los responsables del negocio para evitar afectaciones adicionales.

La comunicación con los stakeholders es fundamental durante este tipo de incidentes. El equipo debe conocer el impacto, las acciones ejecutadas y el tiempo estimado de recuperación del servicio.

7.3 Postmortem del incidente

Una vez restablecido el servicio, se realiza un análisis postmortem para identificar la causa raíz del incidente. Este análisis permite determinar qué ocurrió, cuándo ocurrió y cuáles fueron las consecuencias sobre la operación.

En el escenario planteado, la causa raíz podría corresponder a una consulta de larga duración que consumió recursos excesivos y generó bloqueos sobre otras operaciones. El impacto podría reflejarse en la indisponibilidad temporal del sistema y el incumplimiento parcial de los acuerdos de nivel de servicio.

Las acciones correctivas podrían incluir la optimización de consultas, la creación de índices adicionales y la actualización de estadísticas. Las acciones preventivas incluirían la implementación de alertas y monitoreo proactivo.

Query

Query History

1

EXPLAIN ANALYZE

2

SELECT *

3

FROM productos

4

WHERE nombre = 'Laptop';

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Showing rows: 1 to 5

Page No:

QUERY PLAN

text

1

Seq Scan on productos (cost=0.00..1.05 rows=1 width=20) (actual time=0.053..0.054 rows=1 loop...

2

Filter: ((nombre)::text = 'Laptop')::text)

3

Rows Removed by Filter: 3

4

Planning Time: 0.513 ms

5

Execution Time: 0.064 ms

La gestión del incidente seguiría las buenas prácticas de ITIL. Inicialmente se registra el incidente y se asigna una prioridad según el impacto y la urgencia. Posteriormente se realiza la investigación del problema, se ejecutan las acciones de recuperación y finalmente se documentan las lecciones aprendidas.

Los cambios implementados para evitar la repetición del incidente se gestionan mediante procesos controlados de gestión del cambio, garantizando que las modificaciones sean evaluadas, aprobadas y documentadas antes de su implementación en producción.

8. BASE DE DATOS EN LA NUBE E INFRAESTRUCTURA HÍBRIDA

8.1 Configuración y gestión de una base de datos en la nube

/ Welcome to Neon

Now, let's create your first project.

Project name

empresa_ventas

Postgres version

16

Region

AWS US East 1 (N. Virginia)

Backend Services

☐ Neon Auth

Adds ready-to-use authentication to your app — users and sessions are stored directly in your database. [Learn more](#)

Connect to your database

Branch

production Default

Compute

Primary Active

Database

neondb

Role

neondb_owner

[Reset password](#)

Connection string

☒ Connection pooling

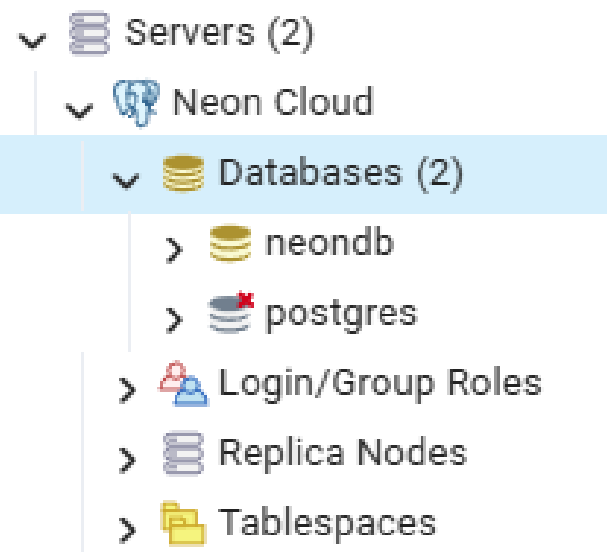
```
postgresql://neondb_owner:*****@ep-lingering-breeze-at6mbvbk-pooler.c-9.us-east-1.aws.neon.tech/neondb?sslmode=require&channel_binding=require
```

 Copy snippet

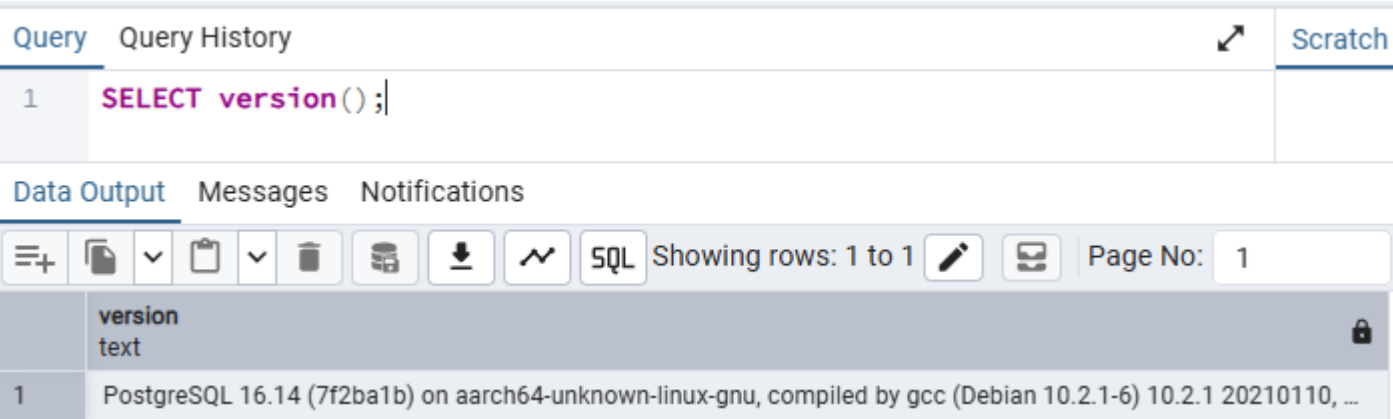
 Show password

Your password is saved in a secure storage vault. [Learn more about connecting](#)

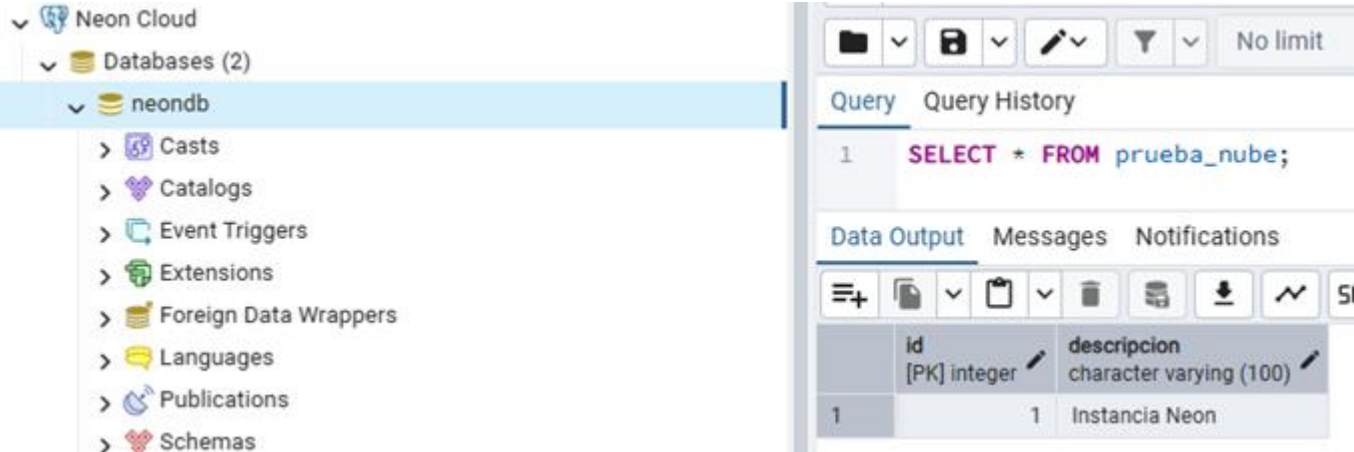
Se implementó una instancia PostgreSQL administrada mediante la plataforma Neon. El servicio proporciona acceso remoto seguro, administración del motor y almacenamiento gestionado en la nube.



Se realizaron consultas de validación sobre la instancia administrada, confirmando el acceso, la conectividad y el correcto funcionamiento de la base de datos desplegada en la nube.



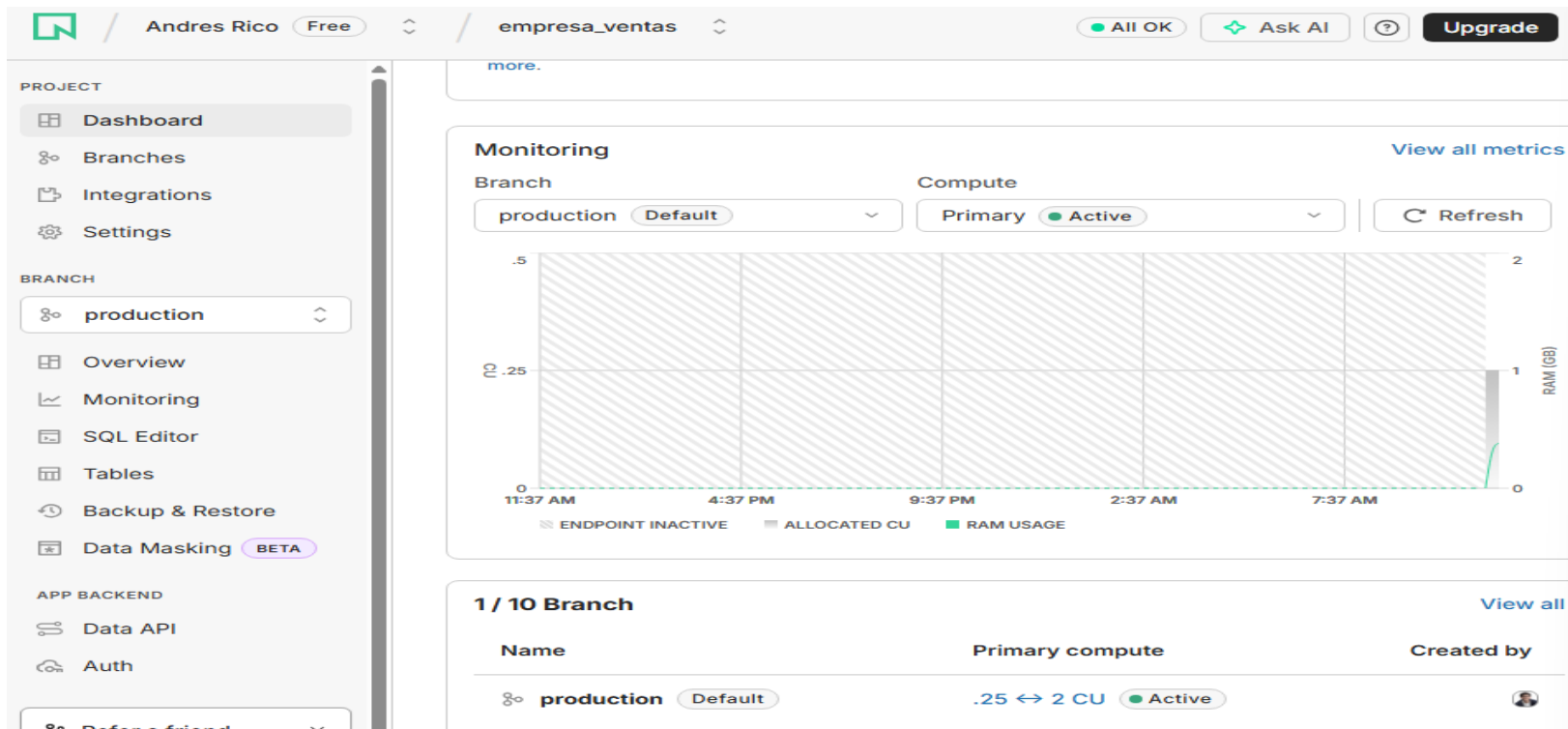
La administración de la base de datos se realizó desde pgAdmin 4, permitiendo ejecutar consultas, administrar objetos y supervisar el entorno de forma remota.



8.2 Comparación entre bases de datos On-Premise y servicios administrados

Las bases de datos instaladas localmente (On-Premise) ofrecen un mayor nivel de control sobre la infraestructura, la configuración del servidor y las políticas de seguridad. La organización administra directamente el hardware, las actualizaciones, los respaldos y las actividades de mantenimiento. Este modelo resulta adecuado cuando existen requisitos regulatorios, políticas estrictas de seguridad o necesidad de personalizar la infraestructura.

Los servicios administrados en la nube, como AWS RDS, Azure SQL o Cloud SQL, delegan gran parte de las tareas operativas al proveedor. Funciones como respaldos automáticos, alta disponibilidad, actualizaciones y monitoreo son administradas por la plataforma, permitiendo que el equipo DBA se enfoque en actividades de optimización y soporte al negocio.



Característica	On-Premise	Nube Administrada
Control de infraestructura	Alto	Medio
Administración del servidor	Propia	Proveedor
Escalabilidad	Limitada	Alta
Alta disponibilidad	Manual	Integrada
Backups	Administrados localmente	Automáticos
Costos iniciales	Altos	Bajos
Mantenimiento	Interno	Proveedor

Las soluciones on-premise serían recomendables en organizaciones con requerimientos regulatorios estrictos, necesidades de personalización o restricciones relacionadas con la ubicación de los datos. Por otra parte, los servicios administrados resultan apropiados para organizaciones que requieren escalabilidad, alta disponibilidad y reducción de la carga operativa.

Desde la perspectiva de costos, las soluciones locales requieren inversión en servidores, almacenamiento, licenciamiento y mantenimiento. Los servicios administrados funcionan bajo esquemas de pago por consumo, reduciendo la inversión inicial y facilitando el crecimiento de la infraestructura según la demanda.

8.3 Seguridad y acceso en un entorno híbrido

En un entorno híbrido donde coexisten bases de datos locales y servicios en la nube, la seguridad debe abordarse mediante múltiples capas de protección. El acceso entre los entornos debe realizarse mediante redes privadas, VPN o túneles seguros, evitando la exposición directa de las bases de datos a Internet.

Las comunicaciones entre clientes y servidores deben protegerse mediante protocolos de cifrado SSL/TLS. El cifrado en tránsito protege la información mientras viaja por la red, mientras que el cifrado en reposo protege los archivos almacenados en discos o servicios de almacenamiento.

Connect to your database

Branch

production

Default

Compute

Primary

Active

Database

neondb

Role

neondb_owner

[Reset password](#)

Connection string

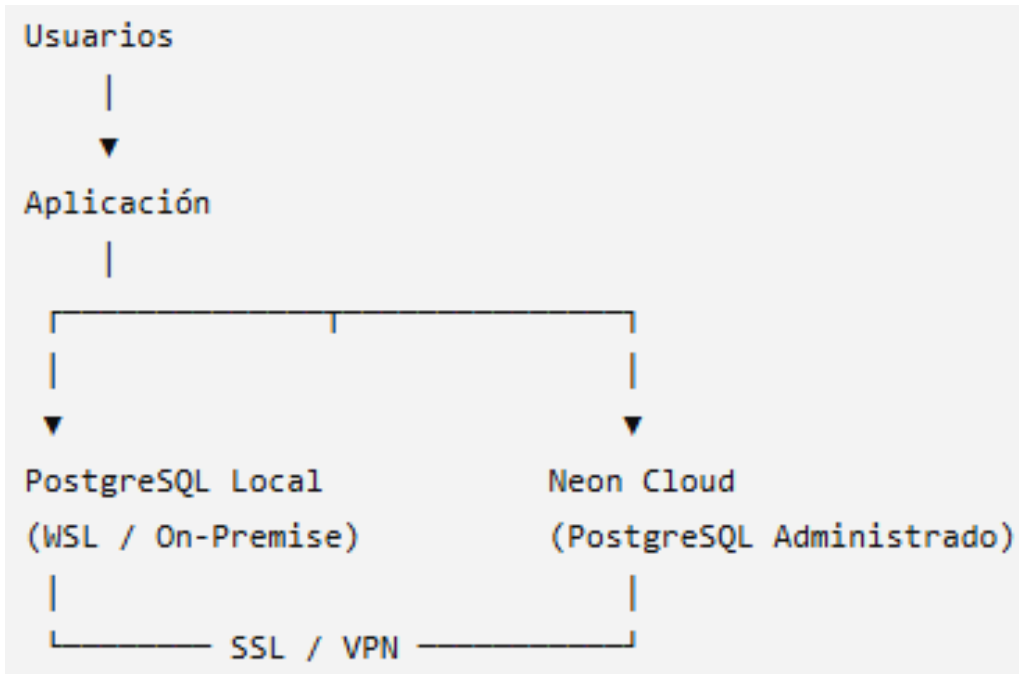
Connection pooling

postgresql://neondb_owner:*****@ep-lingering-breeze-at6mbvbk-pooler.c-9.us-east-1.aws.neon.tech/neondb?sslmode=require&channel_binding=require

La segmentación de la red mediante firewalls, listas de control de acceso y grupos de seguridad permite limitar el acceso únicamente a los servidores y aplicaciones autorizadas. Estas restricciones reducen la superficie de ataque y mejoran la seguridad de la infraestructura.

La gestión de credenciales debe realizarse mediante herramientas especializadas de administración de secretos, evitando almacenar usuarios y contraseñas dentro del código o los scripts. Soluciones como AWS Secrets Manager, Azure Key Vault o HashiCorp Vault permiten centralizar, rotar y proteger las credenciales utilizadas por las aplicaciones y los administradores.

Arquitectura propuesta



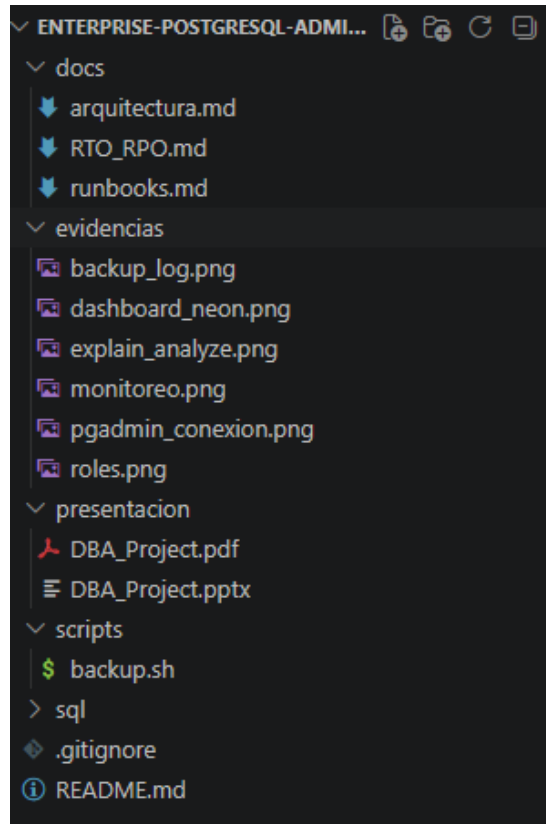
Conclusión

La combinación de infraestructura local y servicios administrados permite aprovechar las ventajas de ambos modelos. Mientras la infraestructura local ofrece mayor control y personalización, la nube proporciona escalabilidad, alta disponibilidad y automatización de tareas administrativas. La selección de la arquitectura adecuada dependerá de los requisitos de seguridad, disponibilidad, costos y necesidades del negocio

9. DOCUMENTACIÓN Y MEJORA CONTINUA

9.1 Mantener documentación actualizada

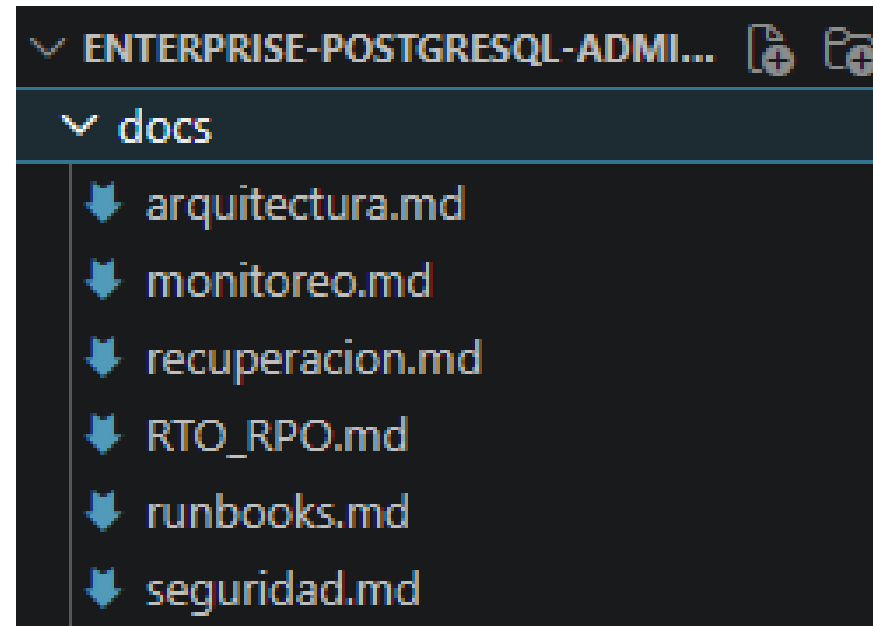
Durante el desarrollo del proyecto se mantuvo documentación actualizada de los procedimientos, configuraciones y decisiones técnicas realizadas sobre la base de datos. La estructura del repositorio permite centralizar scripts SQL, procedimientos almacenados, automatizaciones, documentación y evidencias del entorno implementado.



Los procedimientos operativos fueron documentados mediante runbooks, incluyendo tareas de respaldo, recuperación, monitoreo y mantenimiento. Esta documentación permite reproducir las actividades administrativas y reducir la dependencia del conocimiento individual.

```
runbooks.md > abc # Runbook Operativo > abc ## Analizar co
1  # Runbook Operativo
2
3  ## Verificar estado del servicio
4
5  sudo service postgresql status
6
7  ## Reiniciar PostgreSQL
8
9  sudo service postgresql restart
10
11 ## Monitorear conexiones
12
13 SELECT * FROM pg_stat_activity;
14
15 ## Ejecutar backup
16
17 ./backup.sh
18
19 ## Restaurar base
20
21 sudo -u postgres psql empresa_ventas
22 < backup.sql
23
24 ## Revisar logs
25
26 cat backup.log
27
28 ## Analizar consultas
29
30 EXPLAIN ANALYZE
```

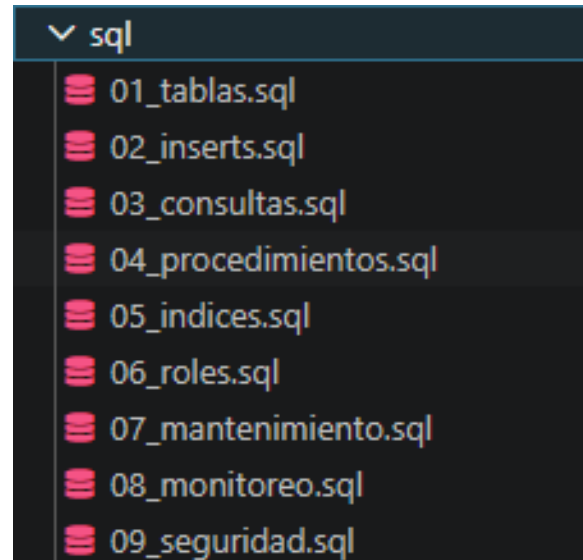
La arquitectura implementada y las estrategias de recuperación fueron documentadas mediante archivos específicos dentro del directorio *docs*, facilitando la consulta y actualización de la información.



9.2 Importancia de documentar cambios

Documentar cambios sobre la base de datos permite mantener la trazabilidad de modificaciones relacionadas con objetos, parámetros, usuarios y configuraciones. Esto facilita la auditoría, la resolución de incidentes y la continuidad operativa.

Los cambios sobre tablas, índices, roles y procedimientos pueden afectar directamente el funcionamiento del sistema, por lo que deben registrarse dentro del repositorio y asociarse a un control de versiones.



Por ultimo mencionar que, la documentación se mantiene viva mediante Git y GitHub, permitiendo registrar versiones, revisar cambios y compartir conocimiento entre los miembros del equipo. Esta práctica facilita la colaboración y asegura que la documentación refleje el estado real del entorno.

Por ultimo mencionar que, la documentación se mantiene viva mediante Git y GitHub, permitiendo registrar versiones, revisar cambios y compartir conocimiento entre los miembros del equipo. Esta práctica facilita la colaboración y asegura que la documentación refleje el estado real del entorno.

GRACIAS, TOTALES